# SELF X SDN control with state embedding memory

*Project report | 20 October 2026*

## Abstract

SELF-X investigates a single software-defined networking controller that retrieves earlier state-action-outcome episodes and their reflections before choosing a flow intervention. The project combines typed JSON decisions, a deterministic action gate, a normalized numeric state embedding, bounded episode memory and measurable feedback. A local website exposes the complete decision trace and provides reproducible experiments.

The evaluation includes a finite-buffer two-path fluid simulation with 2,400 windows across five seeds and four controllers, a paced Gemini 3.5 Flash-Lite pilot, and an OpenRouter free-model integration diagnostic. API calls are real; all network performance values in this report are simulated. A separate Linux Mininet/Ryu adapter is retained but has not been validated as a physical or packet-emulation testbed in this Windows evaluation.

In the offline study, A1 mean modeled latency was 23.86 ms and A2 was 24.98 ms. Memory therefore changed latency by +1.13 ms relative to the same controller without memory. A2 also had 10.69% mean total loss compared with 10.01% for A1. These results do not establish a consistent benefit from memory. They demonstrate why safety, actual API coverage and multiple metrics must accompany the learning claim.

### Report navigation

The following sections cover the problem and research question, related work, architecture, method, experimental protocol, offline results, API pilots, engineering validation, limitations and references. Raw evidence is stored under output/experiments; runnable instructions are in README.md.

# 1 Problem and project objective

Network demand varies over time. A long-lived or bursty high-bandwidth flow can congest a shared bottleneck, increase queueing delay and trigger loss. A static route may continue to use the overloaded path while an alternate path has spare capacity. An administrator needs both an actionable control choice and enough evidence to assess its consequences.

The research question is whether a single controller can improve decisions during repeated traffic episodes by retrieving its own previous experiences. Each experience records a compact state, the installed action, measured outcome, reward and reflection. The later controller can consult similar episodes without retraining its language model.

### Research questions

Can the implementation observe traffic, request a structured decision, reject invalid actions and finish an auditable control cycle?

Does A2 improve latency, delivered goodput or total loss over the otherwise comparable A1 controller?

How do quota errors, malformed responses and action vetoes affect operational results?

Can a reviewer reproduce the traffic and trace a reported metric back to a specific window?

### Intended website behavior

The website is a local research dashboard. It compares controllers, replays an offline experiment from a seed, and exposes the exact decision, safety reason and reflection for each window. It also links the editable report and presentation. Provider credentials remain in Python and are never served to the browser.

The submission focuses only on the selfx folder. Its portable model is designed for a reliable demonstration on this Windows computer. Deployment on a real SDN requires separate testbed instrumentation and validation; a successful local simulation is not evidence of deployment performance.

# 2 Related work and comparison boundary

Reflexion uses verbal feedback and episodic memory to guide language agents over repeated attempts [1]. It motivates storing useful explanations beside measured feedback without fine-tuning. Its published evaluation tasks differ from network flow management, so its task scores cannot be interpreted as an expected SDN improvement.

RouteNet models relationships among topology, routing and traffic using a graph neural network [2]. The paper reports worst-case mean relative error of 15.4% in its KPI prediction evaluation. This is prediction error under that study's data and split, not a latency reduction and not a baseline reproduced by SELF-X. Its results support the relevance of topology-aware modeling but require a separate implementation and dataset for a fair numerical comparison.

| Approach | Role in this report | Comparable numerical result |
|---|---|---|
| Static primary path A0 | Local no-control baseline | Measured on identical traffic |
| Greedy route B1 | Local deterministic baseline | Measured on identical traffic |
| Reflexion | Conceptual basis for episode reflections | No cross-task score comparison |
| RouteNet | Published learned network KPI model | 15.4% MRE belongs to original study |
| Gemini and OpenRouter | Live decision providers | Coverage and local simulated outcome |

The local baselines require no model training and make no claim to reproduce a particular paper's routing algorithm. B1 moves the elephant flow to the less-loaded path only when the alternate remains within the safety ceiling. It provides a simple operational comparator. Offline A1/A2 evaluate deterministic threshold and case-retrieval policies; API pilots evaluate actual provider integration separately.

Official Gemini documentation identifies the requested model endpoint and explains structured output and account-specific rate limits [3-5]. OpenRouter documents its free router and routed model selection [6]. Ryu documents the flow and meter REST interface used by the separate Linux adapter [7]. All external sources were accessed on 20 October 2026.

# 3 System architecture

The portable controller follows an explicit observe-retrieve-decide-validate-act-measure-reflect-store loop. Shared schemas, prompts, memory, reward and safety modules support the portable runner and original Ryu-oriented LangGraph pipeline. The simulator orchestrates these modules directly; the original testbed uses LangGraph nodes.

| Stage | Input | Output |
|---|---|---|
| Observe | Seeded demand and current queues | State summary and numeric features |
| Retrieve | Current embedding and earlier A2 records | Up to four similar episodes |
| Decide | State plus examples | Typed action and rationale |
| Validate and act | Decision and known topology | Allowed intervention or visible no-op |
| Measure | Demand and selected action | Delay goodput loss and reward |
| Reflect and store | Outcome and reward | Explanation plus bounded episode |

## Topology and action lifetime

The portable topology represents primary s1-s2-s4 and alternate s1-s3-s4 logical paths. Each path has a 10 Mbps bottleneck, a 0.5 Mbit queue and an 8 ms propagation component. Background traffic occupies each path independently. One elephant flow starts on the primary in each five-second window. An intervention applies only to that window; queues carry forward within the condition and seed.

The original Mininet topology is linear and has no alternate route. The Ryu graph therefore requires an explicitly validated egress mapping before a reroute is allowed. The portable diamond is a simulation abstraction, not eight instantiated hosts or a deployed OpenFlow topology.

## Navigation and data flow

README.md is the entry point. docs/ARCHITECTURE.md and its R Markdown companion map every module and contain flow diagrams. The project guide explains the purpose and demo. Experiment files connect fixed traffic hashes to CSV measurements, JSONL decisions and provider audits. The report and charts are generated from those saved records.

# 4 Embedding memory and deterministic safety

The state embedding contains normalized primary background, alternate background, elephant demand, both queue occupancies and total primary demand, plus a small constant feature. Each vector is normalized to unit length. Cosine similarity ranks retained episodes. This is a hand-designed numeric representation; no neural embedding model or vector database is trained or queried.

A2 retrieves at most four earlier episodes. Each record contains state, installed action, current parameters, reward and reflection. Memory retains at most 30 records and prunes the lowest rewards. A1 disables record persistence and retrieves no examples. Every seed and condition starts empty, preventing future-window and cross-run leakage.

Offline A1 uses an 85% primary-load trigger. If the alternate can accept the whole elephant flow without exceeding 90% utilization, it reroutes; otherwise it may cap the elephant at 80% of offered demand. Offline A2 can reuse a positively rewarded action from a state with cosine similarity above 0.97, rebuilding parameters for current demand. The independent safety gate still runs. API A1/A2 instead request provider decisions with the same prompt apart from memory examples.

| **Check** | **Enforced behavior** |
|---|---|
| Action schema | Only reroute rate_limit or no_op with required parameters |
| Target validity | Exact known switch flow match and alternate egress |
| Rate reduction | At most 20% of the current elephant demand |
| Meter floor | At least 512 kbps |
| Alternate capacity | Projected offered load no more than 90% of 10 Mbps |
| Failure handling | Record the reason and apply no_op |

These local constraints bound specific interventions. They do not prove global stability, fairness, SLA compliance or end-to-end loss limits. Similarity can retrieve an action inappropriate for the present state, and reward-based pruning can discard useful negative examples. The experiment records vetoes to expose those weaknesses.

# 5 Experimental protocol and network model

| Setting | Main portable run | Gemini pilot | Free-router diagnostic |
|---|---|---|---|
| Seeds | 5 | 3 | 1 |
| Windows per seed and condition | 120 | 24 | 12 |
| Simulated seconds each | 600 | 120 | 60 |
| Controllers | A0 B1 A1 A2 | A0 B1 A1 A2 | A0 B1 A1 A2 |
| Network provenance | Simulation | Simulation | Simulation |
| A1 A2 decisions | Deterministic | Real Gemini or marked fallback | Real free router or marked fallback |

The workload repeats light, primary-burst, alternate-busy and dual-congestion episodes with seeded uniform demand perturbations. It is synthetic aggregate NetFlow-style data rather than a captured traffic dataset. Every condition uses identical demand bytes and starts with zero queues and empty memory. SHA256 hashes in each summary bind the workload to its results.

### Fluid equations

For each path, q_next = min(B, max(0, q_previous + (admitted_rate - C)T)). Overflow is max(0, q_previous + (admitted_rate - C)T - B). Delivered volume is old queue plus admitted volume minus the new queue and overflow. C is 10 Mbps, B is 0.5 Mbit and T is 5 seconds.

The exact average fluid queue is integrated over the window, including saturation or emptying. Delay equals 8 ms plus average queue/C converted to milliseconds, weighted by delivered volume across paths. Goodput is delivered volume/T. Loss includes overflow and deliberate meter drops against all offered volume. The result is a modeled delay proxy, not measured packet RTT, jitter or p95.

### Reward and uncertainty

Reward = -0.4(delta latency/50) + 0.4(delta goodput/5000) - 0.2(delta loss/20), using ms, kbps and percentage points. A same-window no-op counterfactual uses the same starting queue and demand; only the chosen action commits the next queue. Paired seed means, rather than individual windows, define the 95% Student-t intervals. With one seed no interval is reported. Small-sample intervals are exploratory.

# 6 Main offline results

| Controller | Latency ms | Goodput Mbps | Mean loss % | Vetoes |
|---|---|---|---|---|
| A0 | 30.296 | 13.326 | 9.966 | 0 |
| B1 | 27.423 | 13.399 | 9.416 | 0 |
| A1 | 23.859 | 13.308 | 10.013 | 0 |
| A2 | 24.984 | 13.233 | 10.691 | 62 |

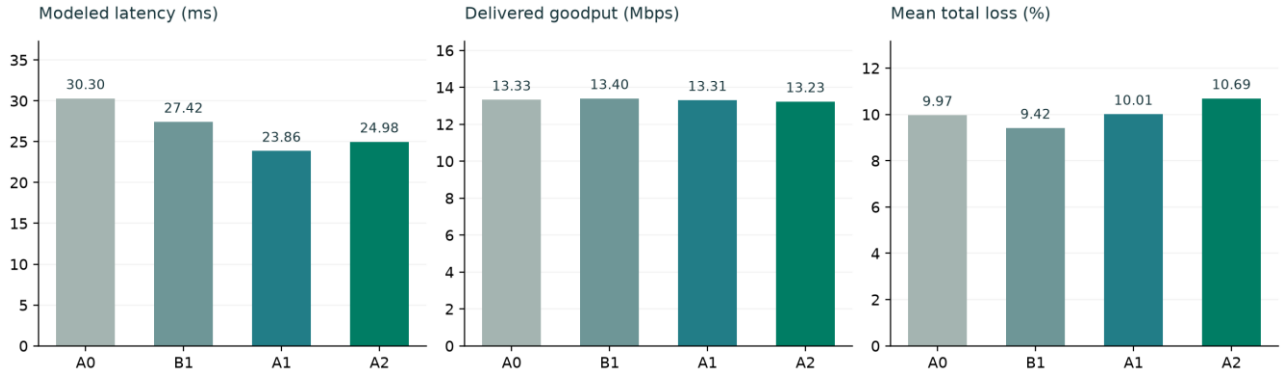



**Figure 1. Five-seed offline simulation. All mean loss values include deliberate meter drops. Each controller has 600 evaluated windows.**

A2 reduced mean modeled latency relative to A0 from 30.30 to 24.98 ms, but A1 was lower at 23.86 ms. A2 had 62 vetoes; similar-state reuse sometimes proposed an action that was not feasible under current demand. The safety gate converted these actions to no-op.

A2 goodput was 13.233 Mbps, compared with 13.308 Mbps for A1. Mean total loss rose from 10.013% to 10.691%. A lower delay alone therefore cannot establish a better overall controller. B1 achieved the lowest total loss among these four means while A1 achieved the lowest delay.

| Paired comparison | Mean delay difference ms | 95 percent seed interval |
|---|---|---|
| B1 minus A0 | -2.874 | [-4.176, -1.571] |
| A1 minus A0 | -6.437 | [-8.723, -4.152] |
| A2 minus A0 | -5.312 | [-7.055, -3.569] |

These intervals quantify variability across five seeds in one synthetic workload family. They do not capture topology variation, model mismatch or deployment uncertainty. Offline decisions are deterministic and cannot establish a language model benefit.

# 7 Reflection and memory findings

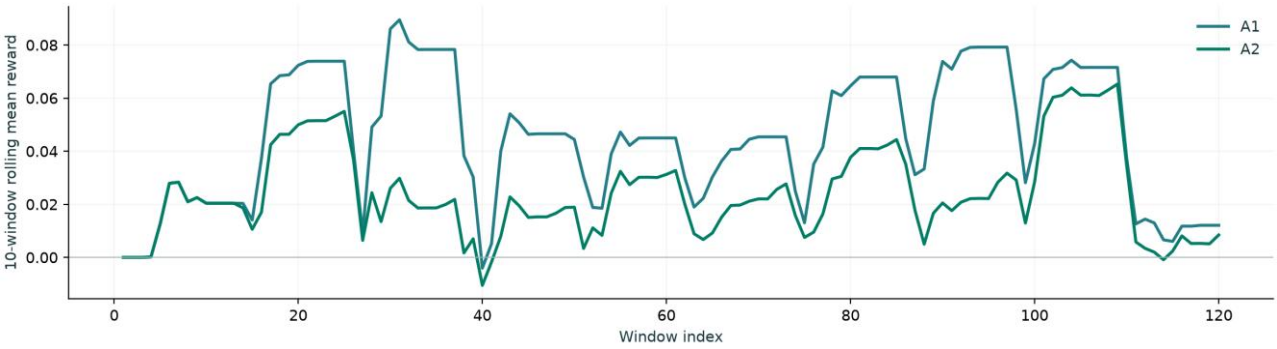



**Figure 2. Ten-window rolling reward averaged over seeds. Reward compares each action with its same-window no-op counterfactual.**

Mean reward was 0.0450 for A1 and 0.0240 for A2. The rolling curve is not evidence of sustained improvement simply because it fluctuates above zero. The central comparison is the controller with memory against the same controller without memory, with goodput and total loss preserved.

Three mechanisms can explain the observed weakness. Cosine similarity over a small feature vector may fail to distinguish states near a feasibility boundary. Reward-based pruning favors previously successful episodes while dropping failures that could warn the controller. Reusing an action label from a related state does not guarantee that the action has the same effect under current path demand.

The safety gate prevents invalid reroutes from reaching the simulator, but vetoes still consume a decision opportunity. A possible next experiment would filter retrieved actions by present feasibility, retain informative negative examples, and compare reward-ranked, similarity-ranked and recent-window memories. Those changes require a fresh preregistered evaluation; their expected benefit is a hypothesis, not a result in this submission.

### Meaning of self improvement

The implementation supports adaptation through context. It does not change model weights, train a new embedding or establish monotonic learning. A complete execution trace demonstrates that the loop works technically. Whether it improves control quality is an empirical question, and this offline result does not support a general improvement claim.

# 8 Live Gemini integration pilot

| Controller | Latency ms | Goodput Mbps | Mean loss % | Vetoes |
|---|---|---|---|---|
| A0 | 30.006 | 13.362 | 11.261 | 0 |
| B1 | 28.194 | 13.428 | 10.772 | 0 |
| A1 | 28.201 | 13.428 | 10.772 | 48 |
| A2 | 28.185 | 13.428 | 10.772 | 22 |

The paced pilot requested gemini-3.5-flash-lite. A1 received API decisions in 72 of 72 windows; A2 received API decisions in 72 of 72 windows. The remaining windows are explicit safe fallbacks. Every actual model identifier, duration, response status and returned usage field is stored in provider_calls.json. Network results remain simulated.

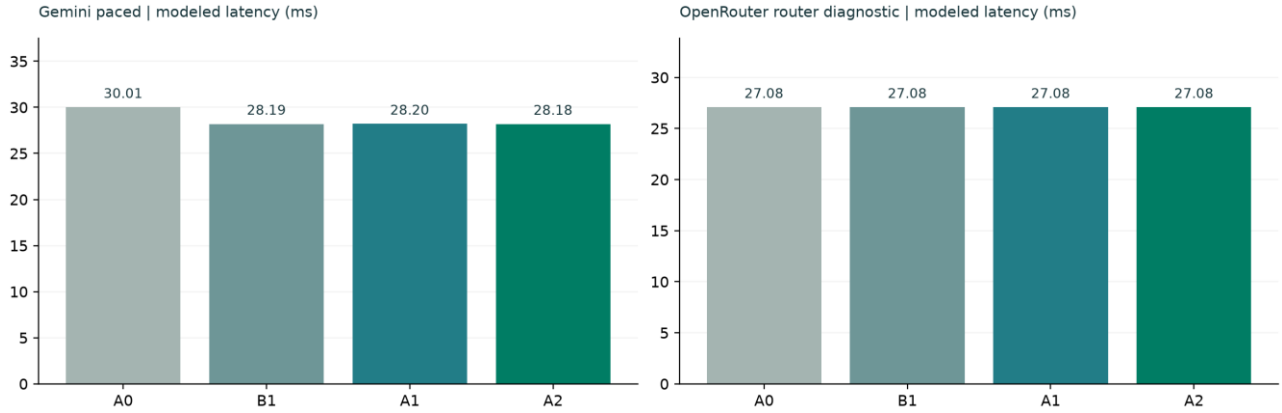



**Figure 3. API pilot mean modeled latency. Different pilot sizes and failure coverage prevent a fair cross-provider performance ranking.**

Gemini A1 mean modeled latency was 28.201 ms and A2 was 28.185 ms. A1 mean total loss was 10.772% and A2 was 10.772%. These values include all attempted operational windows. A successful model response can still be vetoed, and a valid no-op decision need not improve a congested network.

An earlier unpaced run hit HTTP 429 and is retained as an availability diagnostic in a separate directory. The corrected adapter spaces request starts, caps response wait time and opens a circuit breaker after consecutive failures. The paced run uses its own output directory; failed evidence has not been overwritten. Provider quotas depend on project and model [5], so request spacing does not guarantee availability or a free-tier entitlement.

The paced run completed 288 successful provider requests. Its measured wall time was 1211.7 seconds. Request spacing and two calls per control cycle mean that the live API loop is slower than a five-second real-time deadline. Simulation time advances independently; this pilot does not certify real-time control.

# 9 Free models and API availability

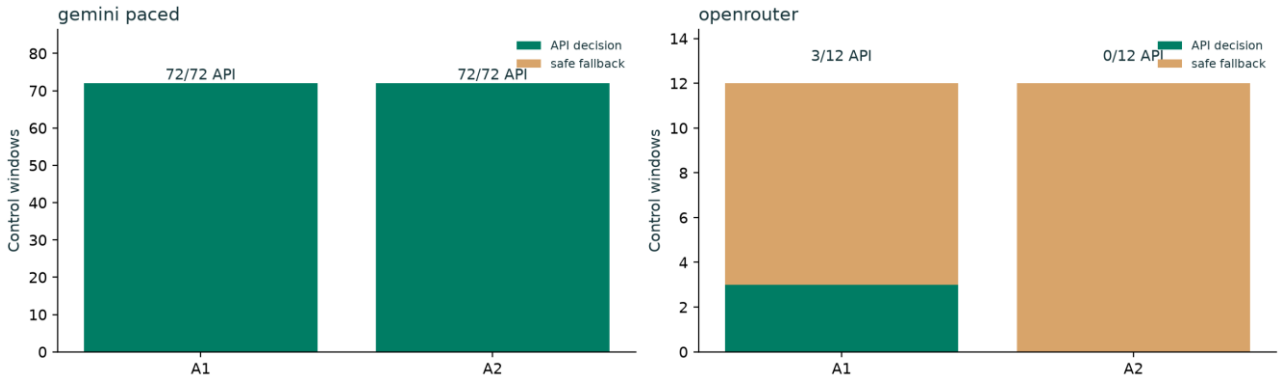



**Figure 4. API decision coverage. Safe fallbacks are counted separately and remain in the performance results.**

OpenRouter's public catalog exposed 17 zero-price model IDs ending in :free. One bounded healthy-network decision was attempted for each ID. 10 returned valid typed decisions and 7 were unavailable or unsuitable in that check. Failure codes include rate limits, access restrictions and non-JSON or incomplete output. A healthy-network check tests access and schema handling; it does not measure congestion-control competence.

The free-router diagnostic received API decisions in 3/12 A1 windows and 0/12 A2 windows. It exposed a practical limitation: a router can select models with different response behavior, including reasoning outputs that do not produce a usable JSON object within the configured token budget. Those requests became visible fallbacks. The actual underlying model and returned cost are recorded for every successful call.

The application blocks paid OpenRouter model IDs. Free routing is useful for a low-cost integration demo, but changing model identity and variable availability confound a controlled model-quality comparison. The diagnostic is therefore reported as operational evidence rather than a proof that one provider or memory strategy performs better.

A fixed-model follow-up should select one accessible free model, verify its output budget on a small case, and evaluate A1/A2 on multiple identical seeds with sufficient successful decision coverage. No reported literature score substitutes for this local experiment.

# 10 Engineering validation and reproducibility

Regression checks cover seeded trace equality, conservation of traffic volume, throughput and queue bounds, invalid targets, alternate overload, embedding retrieval, atomic memory persistence and explicit API failure provenance. The original graph, reward, counters, safety and validated result bundle tests remain part of the suite.

Dashboard checks verify loopback operation, saved comparisons, a working offline replay, individual decision navigation and mobile layout. The HTTP server serves a fixed route allowlist. Requests for .env, source configuration, repository metadata and traversal paths are rejected. Replay requires a same-origin request and accepts bounded integer parameters.

## Corrections to the supplied project

The original result writer defaulted outside selfx; its output now stays under output/testbed_results.

A meter-only OpenFlow override omitted forwarding. The graph now preserves an observed output action and rejects a meter action when forwarding cannot be preserved.

The linear testbed offered no alternate path. Rerouting now requires a tested egress mapping rather than trusting a model-generated port.

Python test imports assumed a parent-level package. They now run from the requested folder.

Gemini and free OpenRouter replace the previous single-provider dependency for current use. Provider failures are sanitized and visible.

## Reproduction sequence

From selfx, run setup.ps1 on a new Windows machine, then run.ps1 test, run.ps1 experiment and run.ps1 dashboard. Existing saved API evidence can be reviewed without making new requests. New live runs require valid credentials and remaining provider quota. The README gives Python equivalents and exact output paths. Rebuilding report content uses scripts/build_artifacts.py; presentation authoring uses the Artifact Tool build module.

# 11 Limitations and next validation

The strongest supported conclusion is that SELF-X provides a traceable local control loop with real provider integration and bounded interventions. The offline memory variant did not consistently outperform its memory-free comparator. The API pilots have a small sample size and operational failures, which limit claims about model quality and memory benefit.

The fluid model uses constant demand within five-second windows, fixed path capacity and a simplified queue-delay proxy. It does not implement packet scheduling, TCP congestion control, real NetFlow collection, propagation variation, OpenFlow control-plane delay, link failures or real hardware. All reported path-performance measurements are simulation values. No external traffic dataset calibrates this model.

The numeric embedding is manually constructed. Cosine similarity does not ensure that a retrieved episode is causally relevant, and reward pruning can bias the memory. The reward weights encode a selected tradeoff among latency, goodput and loss; changing them can change the preferred action. A formal stability or SLA guarantee is not established.

The Windows evaluation cannot certify Linux Mininet, Open vSwitch, Ryu, iperf3 or root networking. The retained linear topology needs endpoint instrumentation for goodput and end-to-end loss; its aggregate switch counters count transmissions across hops. Its installation and sustained 600-second packet-emulation runs remain deployment work, separate from the accelerated simulation.

### Recommended research sequence

First, add endpoint measurements and validate topology-specific action mappings in a dedicated Linux testbed. Next, compare memory retrieval variants with fixed models and adequate API coverage. Then replay an appropriately licensed captured workload and vary topology, link capacity and congestion patterns. Report matched baselines and uncertainty without dropping failures. Only those studies can support broader claims about real-network performance or generalization.

# References and evidence files

[1] Shinn et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS. https://arxiv.org/abs/2303.11366

[2] Rusek et al. (2019). RouteNet: Leveraging Graph Neural Networks for network modeling and optimization in SDN. https://arxiv.org/abs/1910.01508

[3] Google. Gemini API model catalog. https://ai.google.dev/gemini-api/docs/models

[4] Google. Structured outputs. https://ai.google.dev/gemini-api/docs/structured-output

[5] Google. Gemini API rate limits. https://ai.google.dev/gemini-api/docs/rate-limits

[6] OpenRouter. Free Models Router. https://openrouter.ai/docs/guides/routing/routers/free-router

[7] Ryu. ofctl_rest reference. https://ryu.readthedocs.io/en/latest/app/ofctl_rest.html

Web sources accessed 20 October 2026. Reported RouteNet prediction error belongs to its original study and is not reproduced or compared as an SDN control improvement here.

### Local evidence

output/experiments/offline: five-seed deterministic study, CSV metrics, JSONL trace and traffic hashes.

output/experiments/gemini_paced: paced Gemini pilot and actual provider audit.

output/experiments/gemini: unpaced quota diagnostic retained separately.

output/experiments/openrouter: free-router integration diagnostic.

output/provider_check.json and output/free_model_checks.json: authenticated connectivity and complete discovered free-model sweep.

docs/ARCHITECTURE.md, docs/ARCHITECTURE.Rmd and docs/EXPERIMENTS.md: module flow and exact protocol.